

StockSense

Smart Inventory Management System

Product Requirements Document (PRD)

Implementation specification for an AI coding IDE

Frontend: React Backend: Node.js + Express ORM: Prisma Database:
PostgreSQL

This document converts the provided StockSense hackathon problem statement into an implementation-ready specification. Requirements explicitly stated in the source are preserved; technical implementation details are proposed decisions for the project.

Version 1.0 • September 2026

1. Product Overview

StockSense is a modular Inventory Management System (IMS) designed to digitize and streamline stock-related operations. The system replaces manual registers, Excel sheets, and scattered tracking methods with a centralized, real-time, easy-to-use application.

The source problem statement identifies Inventory Managers and Warehouse Staff as the target users. Inventory Managers manage incoming and outgoing stock; Warehouse Staff perform transfers, picking, shelving, and counting. The authentication flow is sign up/log in, OTP-based password reset, then redirect to the Inventory Dashboard. Source requirements: [filecite]turn0file0[L2-L10]

Primary product principle: every stock-changing operation must produce a consistent inventory state and an auditable Stock Ledger entry.

1.1 Goals

- Centralize product, stock, warehouse, receipt, delivery, transfer, and adjustment data.
- Provide location-aware inventory quantities.
- Make incoming and outgoing stock operations traceable.
- Maintain an auditable stock ledger for stock movements.
- Provide a dashboard with operational KPIs and filters.
- Support multi-warehouse inventory.
- Provide low-stock alerts and SKU search.
- Keep the architecture simple enough for rapid hackathon development while maintaining transaction-safe stock updates.

1.2 Non-Goals for the MVP

- Full accounting, invoicing, purchasing, sales CRM, or ERP functionality.
- Payment processing.
- External inventory/product catalogs.
- Microservices, Kafka, Kubernetes, or other distributed infrastructure unless later required.
- Native mobile application.
- SMS OTP; the selected MVP uses email OTP through Brevo SMTP.

1.3 Source-to-Implementation Rule

The provided problem statement is the authoritative source for product behavior. Technical choices such as Express, Prisma, database schema, JWT, Nodemailer, Brevo SMTP, and API structure are implementation decisions for the AI IDE and should not be presented as requirements from the hackathon document.

2. Technology Stack & Environment

Layer	Technology	Purpose
Frontend	React	Inventory dashboard, forms, tables, filters, authentication UI
Backend	Node.js + Express	REST API, business logic, authentication, stock workflows

Layer	Technology	Purpose
ORM	Prisma	Database access, schema, migrations, transactions
Database	PostgreSQL	Users, products, stock, operations, ledger, notifications
Email	Brevo SMTP + Nodemailer	Password-reset OTP delivery
Authentication	JWT + bcrypt	Session/access-token authentication and password hashing
Version control	Git + GitHub	Source control and team collaboration
Deployment	Cloud frontend + cloud backend + managed PostgreSQL	Deployment to production

2.1 Local + Cloud Strategy

Development should support a local PostgreSQL instance while deployment uses a cloud PostgreSQL database. Prisma should read the database connection from DATABASE_URL, allowing the same application code to work in both environments.

```
# .env.example
DATABASE_URL="postgresql://USER:PASSWORD@HOST:5432/stocksense"
JWT_SECRET="replace-with-a-long-random-secret"
BREVO_SMTP_HOST="smtp-relay.brevo.com"
BREVO_SMTP_PORT="587"
BREVO_SMTP_USER="your-brevo-smtp-login"
BREVO_SMTP_PASSWORD="your-brevo-smtp-key"
BREVO_SENDER_EMAIL="verified-sender@example.com"
BREVO_SENDER_NAME="StockSense"
FRONTEND_URL="http://localhost:5173"
PORT="5000"
NODE_ENV="development"
```

Security: .env must never be committed. Provide .env.example with variable names only. Brevo credentials must exist only in the backend.

3. User Roles & Access Model

Role	Core Responsibilities	MVP Access
Inventory Manager	Manage products, incoming/outgoing stock, adjustments	Full warehouse inventory, dashboard
Warehouse Staff	Picking, shelving, transfers, counting, operational stock	Optional inventory access
Authenticated User	Profile and logout	Own profile/authentication

The source identifies two target user groups. The exact permission matrix is not specified in the source; therefore the AI IDE should implement a simple role field and a centralized authorization middleware so permissions can be tightened later without redesigning the database.

4. Functional Requirements

4.1 Authentication

- User can sign up with required identity fields and password.
- User can log in with email and password.
- Passwords must be hashed using bcrypt; never store plaintext passwords.
- Successful login returns an access token and redirects the frontend to the dashboard.

- User can request a password-reset OTP by email.
- OTP is short-lived, single-use, stored hashed, and protected by an attempt limit and resend cooldown.
- User can verify a valid OTP and set a new password.
- User can log out; frontend clears the access token/session.
- Protected API routes require authentication.

4.2 Dashboard

The landing page after authentication must show the required operational snapshot.

KPI	Definition
Total Products in Stock	Number of products with positive total available stock across locations
Low Stock / Out of Stock	Products at or below their reorder threshold; separately identify zero stock
Pending Receipts	Receipts not yet completed/validated
Pending Deliveries	Deliveries not yet completed/validated
Internal Transfers Scheduled	Transfers that are pending/ready and not completed

Dashboard filters must support document type (Receipts, Delivery, Internal, Adjustments), status (Draft, Waiting, Ready, Done, Canceled), warehouse/location, and product category. These filters are explicitly required by the source. [filecite turn0file0 L11-L23](#)

4.3 Products

- Create and update products.
- Fields: name, SKU/code, category, unit of measure, optional initial stock.
- Maintain reorder level/rule for low-stock detection.
- Search products by SKU/name.
- Show stock availability per location.
- Prevent duplicate SKU values.

The product fields and product capabilities come directly from the source requirements.

[filecite turn0file0 L26-L31](#) [filecite turn0file0 L43-L50](#)

4.4 Receipts — Incoming Goods

Receipts are used when items arrive from vendors. Required flow: create receipt → add supplier and products → enter quantities → validate → stock increases automatically.

Receipt status flow:
DRAFT → READY → DONE
↘ CANCELED

Validation rule:
For each receipt line:
location stock += received quantity
stock ledger += RECEIPT / IN
receipt status = DONE

Example: receiving 50 units of Steel Rods increases stock by 50. [filecite turn0file0 L51-L59](#)

4.5 Delivery Orders — Outgoing Goods

Delivery orders are used when stock leaves the warehouse for customer shipment. The source describes pick → pack → validate, after which stock decreases automatically.

```
Delivery status flow:
DRAFT → READY → DONE
↳ CANCELED

Validation rule:
For each delivery line:
verify sufficient available stock
location stock -= delivered quantity
stock ledger += DELIVERY / OUT
delivery status = DONE
```

Example: delivering 10 chairs reduces available chair stock by 10. [filecite\[turn0file0\]L61-L68](#)

4.6 Internal Transfers

Internal transfers move stock between company locations. Total company stock remains unchanged, but source and destination location quantities change. Every movement must be logged.

```
Transfer validation:
source location stock -= quantity
destination location stock += quantity
stock ledger records OUT from source and IN to destination
transfer status = DONE
```

The source gives examples including Main Warehouse → Production Floor, Rack A → Rack B, and Warehouse 1 → Warehouse 2. [filecite\[turn0file0\]L69-L75](#)

4.7 Stock Adjustments

Adjustments reconcile recorded stock with physical counts. User selects product/location and enters counted quantity. The system computes the delta, updates stock, and logs the adjustment.

```
delta = countedQuantity - recordedQuantity

if delta > 0:
stock += delta
else:
stock -= abs(delta)

ledger += ADJUSTMENT(delta)
```

Example from the source: 3 kg of damaged steel produces a -3 stock adjustment. [filecite\[turn0file0\]L76-L83](#)

4.8 Stock Ledger

The ledger is the audit trail for inventory movements. It must be append-oriented: completed stock movements create ledger entries, while ordinary reads never alter ledger history.

Field	Purpose
id	Unique ledger entry
productId	Product affected
warehouseId	Warehouse context
locationId	Location context
operationType	RECEIPT, DELIVERY, TRANSFER_IN, TRANSFER_OUT, ADJUSTMENT
quantity	Signed movement quantity

Field	Purpose
referenceType	Source document type
referenceId	Source document ID
balanceAfter	Optional resulting location balance
createdBy	User who performed/validated action
createdAt	Timestamp

The source explicitly states that movements and adjustments are logged in the Stock Ledger.

filecite turn0file0L93-L102

5. Proposed Data Model

The following schema is an implementation design for PostgreSQL + Prisma. It is intentionally normalized enough for multi-location stock and auditability.

Entity	Important fields
User	id, name, email, passwordHash, role, createdAt, updatedAt
PasswordResetOTP	id, userId, otpHash, expiresAt, attempts, maxAttempts, usedAt, createdAt
Category	id, name, description, createdAt
UnitOfMeasure	id, name, symbol
Product	id, name, sku, categoryId, uomId, reorderLevel, active, timestamps
Warehouse	id, name, code, address, active, timestamps
Location	id, warehouseId, name, code, type, active, timestamps
StockQuantity	id, productId, locationId, quantity, updatedAt
Supplier	id, name, contactName, email, phone, address
Receipt	id, supplierId, warehouseId, status, referenceNo, createdBy, validatedBy, timestamps
ReceiptLine	id, receiptId, productId, locationId, quantity
Delivery	id, warehouseId, status, referenceNo, createdBy, validatedBy, timestamps
DeliveryLine	id, deliveryId, productId, locationId, quantity
InternalTransfer	id, sourceWarehouseId, destinationWarehouseId, status, referenceNo, createdBy, validatedBy, timestamps
TransferLine	id, transferId, productId, sourceLocationId, destinationLocationId, quantity
StockAdjustment	id, warehouseId, locationId, status, reason, createdBy, validatedBy, timestamps
AdjustmentLine	id, adjustmentId, productId, countedQuantity, previousQuantity, delta
StockLedger	id, productId, warehouseId, locationId, operationType, quantity, referenceType, referenceId, balanceAfter,
Notification	id, userId, type, title, message, readAt, createdAt

5.1 Relationship Rules

- One Warehouse has many Locations.
- One Product can exist in many Locations through StockQuantity.

- One Category has many Products.
- One Receipt has many ReceiptLines.
- One Delivery has many DeliveryLines.
- One InternalTransfer has many TransferLines.
- One StockAdjustment has many AdjustmentLines.
- Every stock-changing document references ledger entries through referenceType/referenceId.
- SKU must be unique.
- Product + Location must be unique in StockQuantity.

6. Inventory Consistency & Transaction Rules

Stock correctness is the most important technical requirement. Any operation that changes stock must execute atomically.

6.1 Database Transaction

Example:

BEGIN TRANSACTION

1. Validate document status.
2. Validate product/location relationships.
3. Validate quantities are positive.
4. Lock/read affected StockQuantity rows as required.
5. Validate available stock for outbound operations.
6. Update StockQuantity.
7. Create StockLedger records.
8. Mark source document DONE.
9. Record validatedBy and timestamp.

COMMIT

If any step fails → ROLLBACK all changes.

6.2 Double-Validation Protection

- A document in DONE or CANCELED status cannot be validated again.
- The validate endpoint must be idempotent against repeated requests.
- The backend, not the React frontend, is the authority for stock changes.
- Frontend quantities must never be trusted without backend validation.
- Negative stock should be rejected for deliveries/transfers unless a future business rule explicitly enables it.

7. REST API Specification

7.1 Authentication

Method	Endpoint	Purpose
POST	/api/auth/register	Create user account
POST	/api/auth/login	Authenticate user

Method	Endpoint	Purpose
POST	/api/auth/forgot-password	Request OTP
POST	/api/auth/verify-otp	Verify OTP
POST	/api/auth/reset-password	Set new password using verified reset flow
GET	/api/auth/me	Get current user
POST	/api/auth/logout	Logout/session cleanup if applicable

7.2 Product & Master Data

Method	Endpoint	Purpose
GET	/api/products	List/search/filter products
POST	/api/products	Create product
GET	/api/products/:id	Get product
PUT	/api/products/:id	Update product
GET	/api/categories	List categories
POST	/api/categories	Create category
GET	/api/warehouses	List warehouses
POST	/api/warehouses	Create warehouse
GET	/api/warehouses/:id/locations	List locations
POST	/api/warehouses/:id/locations	Create location
GET	/api/suppliers	List suppliers
POST	/api/suppliers	Create supplier

7.3 Operations

Method	Endpoint	Purpose
GET/POST	/api/receipts	List/create receipts
GET	/api/receipts/:id	Get receipt
PUT	/api/receipts/:id	Edit draft receipt
POST	/api/receipts/:id/validate	Validate and increase stock
POST	/api/receipts/:id/cancel	Cancel receipt
GET/POST	/api/deliveries	List/create deliveries
GET	/api/deliveries/:id	Get delivery
PUT	/api/deliveries/:id	Edit draft delivery
POST	/api/deliveries/:id/validate	Validate and decrease stock
POST	/api/deliveries/:id/cancel	Cancel delivery
GET/POST	/api/transfers	List/create transfers

Method	Endpoint	Purpose
POST	/api/transfers/:id/validate	Move stock between locations
POST	/api/transfers/:id/cancel	Cancel transfer
GET/POST	/api/adjustments	List/create adjustments
POST	/api/adjustments/:id/validate	Apply physical-count adjustment
GET	/api/stock	Location/product stock query
GET	/api/stock/ledger	Ledger history

7.4 Dashboard & Notifications

Method	Endpoint	Purpose
GET	/api/dashboard/summary	KPI summary
GET	/api/dashboard/operations	Filtered operations
GET	/api/notifications	List current user's alerts
PATCH	/api/notifications/:id/read	Mark alert read
PATCH	/api/notifications/read-all	Mark all alerts read

8. OTP Password Reset Specification

8.1 Flow

1. User submits email to POST /api/auth/forgot-password.
2. Backend finds user.
3. Generate a cryptographically secure 6-digit OTP.
4. Hash OTP before database storage.
5. Store expiry (recommended: 10 minutes), attempts, and createdAt.
6. Invalidate previous active OTPs for that user.
7. Send OTP through Nodemailer → Brevo SMTP.
8. User submits OTP.
9. Backend compares hash, checks expiry, usedAt, and attempt limit.
10. Mark OTP as consumed / create short-lived reset authorization.
11. User sets new password.
12. Hash new password and update user.
13. Invalidate remaining reset records.

8.2 Email

Brevo is only the delivery provider. OTP generation, expiration, verification, and database persistence belong to the StockSense backend.

Subject: StockSense Password Reset OTP

Hello {{name}},

Your StockSense password reset OTP is:

{{OTP}}

This OTP expires in 10 minutes and can only be used once.

If you did not request a password reset, ignore this email.

8.3 Security

- Never log the raw OTP.
- Never return the OTP from an API response.
- Rate-limit forgot-password requests.
- Add resend cooldown (e.g. 60 seconds).
- Limit verification attempts (e.g. 5).
- Return a generic response for unknown email addresses to reduce account enumeration.
- Keep Brevo SMTP credentials server-side only.

9. Frontend Information Architecture

Area	Pages / Components
Auth	Login, Register, Forgot Password, Verify OTP, Reset Password
Dashboard	KPI cards, operations table, filters, low-stock section, alerts
Products	Product list, create/edit form, product detail, stock-by-location
Operations	Receipts, Delivery Orders, Internal Transfers, Adjustments, Move History
Warehouse	Warehouse list, location list, warehouse/location setup
Profile	My Profile, Logout
Shared	Sidebar, top bar, breadcrumbs, modal, table, pagination, filters, status badge, confirmation dialog, toast

9.1 Navigation

The source specifies a left-side navigation containing Products, Operations (Receipts, Delivery Orders, Inventory Adjustment, Move History, Dashboard), Settings/Warehouse, and a Profile Menu with My Profile and Logout.

```

Sidebar
├─ Dashboard
├─ Products
├─ Operations
│ └─ Receipts
│ └─ Delivery Orders
│ └─ Inventory Adjustments
│ └─ Move History
├─ Settings
│ └─ Warehouse
├─ Profile
├─ My Profile
└─ Logout
```

9.2 UI Rules

- Use consistent status badges: Draft, Waiting, Ready, Done, Canceled.
- Use confirmation dialogs before irreversible validation/cancellation.
- Show clear stock quantities and locations.

- Disable validation when required lines or quantities are invalid.
- Show backend validation errors in user-friendly form.
- Use loading, empty, and error states for every data page.
- Do not implement business logic solely in React; React should call backend APIs.

10. Backend Architecture

```

backend/
├── src/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── routes/
│   ├── services/
│   ├── repositories/ # optional if useful with Prisma
│   ├── validators/
│   ├── utils/
│   │   ├── auth/
│   │   ├── otp/
│   │   └── mail/
│   ├── app.ts
│   └── server.ts
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── .env
├── .env.example
└── package.json
  
```

10.1 Layering

Route → Controller → Service → Prisma → PostgreSQL. Controllers should remain thin. Business rules such as stock validation and state transitions belong in services. Prisma transactions should wrap stock-changing operations.

10.2 Error Handling

- Use a centralized error middleware.
- Return consistent JSON: { success, message, data?, errors? }.
- Use HTTP 400 for validation/business input errors, 401 for unauthenticated, 403 for forbidden, 404 for missing resources, 409 for conflicts, and 500 for unexpected errors.
- Never expose stack traces or secrets in production responses.

11. Validation & Business Rules

Rule	Behavior
SKU uniqueness	Reject duplicate SKU with 409 conflict
Quantity	Must be numeric and > 0 for receipt/delivery/transfer lines
Delivery stock	Cannot deliver more than available stock
Transfer source stock	Cannot transfer more than available source stock

Rule	Behavior
Transfer locations	Source and destination must differ
Receipt validation	Only validate eligible non-terminal receipt
Delivery validation	Only validate eligible non-terminal delivery
Adjustment	Counted quantity cannot be negative
Terminal documents	DONE/CANCELED documents cannot be edited
Ledger	Every completed stock movement creates ledger records
Stock row	Unique product + location
Email	Normalize email to lowercase

12. Search, Filters & Pagination

The system must support SKU search and smart filters. The source also requires filters by document type, status, warehouse/location, and product category. [filecite\turn0file0\L19-L23](#)

[filecite\turn0file0\L84-L87](#)

Recommended query convention:

GET /api/products?search=steel&categoryId=...&page;=1&limit;=20

GET /api/operations?type=RECEIPT&status;=READY&warehouseId;=...

GET /api/stock?productId=...&warehouseId;=...&locationId;=...

GET /api/stock/ledger?productId=...&operationType;=DELIVERY&page;=1&limit;=50

All list endpoints should return pagination metadata such as page, limit, total, and totalPages.

13. Seed Data & Demo Scenario

The AI IDE should create deterministic seed data so the hackathon demo works immediately after migration and seeding.

- 2 users: one Inventory Manager and one Warehouse Staff.
- 2 warehouses.
- Several locations such as Main Store, Production Rack, Rack A, Rack B.
- Product categories such as Raw Material, Finished Goods, Consumables.
- At least 8–12 products with realistic SKUs and units.
- At least 2 suppliers.
- Initial stock across multiple locations.
- A few low-stock and out-of-stock products to demonstrate alerts.
- Example receipts, deliveries, transfers, and adjustments in different statuses.
- Ledger entries corresponding to completed stock movements.

13.1 Demo Story

Step 1: Receive 100 kg Steel
→ stock increases by 100.

Step 2: Transfer Steel
Main Store → Production Rack

- total company stock unchanged.
- location quantities change.
- transfer ledger records created.

Step 3: Deliver 20 kg

- available stock decreases by 20.
- delivery ledger record created.

Step 4: Adjust 3 kg damaged

- stock decreases by 3.
- adjustment ledger record created.

Dashboard and Move History must reflect all four operations.

This follows the source's simplified inventory example. [filecite\turn0file0\L88-L102](#)

14. Testing Requirements

14.1 Backend Tests

- Authentication: register, login, invalid password, protected route.
- OTP: generate, send, expiry, wrong OTP, attempt limit, reuse prevention, resend cooldown.
- Products: CRUD, duplicate SKU.
- Receipts: create, validate, stock increase, repeated validation rejection.
- Deliveries: insufficient stock rejection, validate, stock decrease.
- Transfers: source decrease + destination increase in one transaction.
- Adjustments: delta calculation and ledger entry.
- Ledger: correct operation type, quantity, reference, and resulting balance.
- Dashboard: KPI values match database state.

14.2 Frontend Tests / QA

- All pages handle loading, success, empty, and error states.
- Forms validate required fields.
- Unauthorized users cannot access protected routes.
- Filters produce expected backend requests.
- Validation confirmations are shown before stock changes.
- Dashboard updates after completed stock operations.

15. Deployment Plan

Use separate environment variables for local and cloud environments. The cloud database must run Prisma migrations before the backend starts serving traffic.

Local:

React → localhost backend → local PostgreSQL

Cloud:

React hosting → cloud Express backend → managed PostgreSQL

↳ Brevo SMTP

15.1 Deployment Checklist

- Create managed PostgreSQL database.
- Set DATABASE_URL on backend host.
- Run Prisma migrations.
- Seed only if appropriate for demo environment.
- Set JWT_SECRET.
- Set Brevo SMTP variables.
- Set FRONTEND_URL/CORS allowed origin.
- Build and deploy React frontend.
- Build/start Express backend.
- Verify health endpoint.
- Test signup, login, OTP reset, receipt, delivery, transfer, adjustment, and dashboard.
- Never expose DATABASE_URL, JWT_SECRET, or Brevo SMTP key to the frontend.

16. Suggested Repository Structure

```
stocksense/  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   ├── pages/  
│   │   ├── layouts/  
│   │   ├── hooks/  
│   │   ├── services/  
│   │   ├── context/  
│   │   ├── utils/  
│   │   └── types/  
│   ├── .env.example  
│   └── package.json  
├── backend/  
│   ├── src/  
│   │   ├── config/  
│   │   ├── controllers/  
│   │   ├── middleware/  
│   │   ├── routes/  
│   │   ├── services/  
│   │   ├── validators/  
│   │   ├── utils/  
│   │   ├── app.ts  
│   │   └── server.ts  
│   ├── prisma/  
│   │   ├── schema.prisma  
│   │   ├── migrations/  
│   │   └── seed.ts  
│   ├── .env.example  
│   └── package.json  
├── README.md  
├── .gitignore  
└── docker-compose.yml # optional local infrastructure
```

17. AI IDE Implementation Instructions

Paste the following operating instructions into the AI IDE before asking it to implement the project.

AI IDE MASTER INSTRUCTION

Build StockSense as a production-structured hackathon MVP using React on the frontend, Node.js + Express on the backend, Prisma ORM, and PostgreSQL. Treat this PRD as the implementation contract. Do not invent unrelated modules. Preserve the required inventory workflows. Implement the backend before relying on frontend mock data. All stock-changing operations must be handled by the backend inside database transactions. Use a modular layered architecture: Route → Controller → Service → Prisma → PostgreSQL. Use JWT authentication and bcrypt password hashing. Use Nodemailer with Brevo SMTP for password-reset OTP delivery. Never expose secrets to the frontend. Store OTP hashes, not raw OTPs. Implement validation, centralized error handling, authorization middleware, pagination, filtering, and consistent API responses. Create Prisma migrations and deterministic seed data. After each major module, run lint/typecheck/tests and fix errors before proceeding. Do not silently change the data model or business rules; document any necessary deviation. Build in phases: (1) project scaffolding, (2) Prisma schema/migrations/seed, (3) auth + OTP, (4) product/master data, (5) stock engine + ledger, (6) receipts, (7) deliveries, (8) transfers, (9) adjustments, (10) dashboard/notifications, (11) React pages/components, (12) integration testing, (13) deployment configuration and README.

17.1 AI IDE Must Not Do

- Do not store stock as a single product-level number when location-level stock is required.
- Do not update stock from the frontend.
- Do not use raw OTPs in database records.
- Do not commit .env or secrets.
- Do not allow repeated validation of a completed operation.
- Do not create fake API responses after the backend is implemented.
- Do not introduce unnecessary microservices or infrastructure.
- Do not remove the Stock Ledger.

18. Definition of Done

Area	Done when...
Auth	Register/login/protected routes work; OTP reset works through Brevo email.
Products	CRUD, categories, UOM, SKU search, reorder levels work.
Warehouse	Multiple warehouses and locations are supported.
Stock	Location-wise stock is correct and transaction-safe.
Receipts	Validation increases stock and writes ledger entries.
Deliveries	Validation decreases stock and prevents insufficient stock.
Transfers	Source decreases, destination increases, total stock remains unchanged.
Adjustments	Physical count creates correct delta and ledger entry.
Ledger	All completed stock movements are auditable.

Area	Done when...
Dashboard	Required KPIs and filters work from real database data.
Alerts	Low-stock conditions appear in the UI.
Frontend	Required navigation, forms, tables, filters, states, and profile/logout work.
Deployment	Cloud frontend/backend/database work with production environment variables.
Documentation	README contains setup, env variables, migrations, seed, run and deployment instructions.

19. Open Decisions / Assumptions

The source problem statement does not define every technical detail. The following are deliberate implementation assumptions and can be changed without altering the core product requirements.

- Backend framework: Node.js + Express.
- ORM: Prisma.
- Authentication: JWT + bcrypt.
- Password-reset OTP delivery: Brevo SMTP via Nodemailer.
- OTP lifetime: 10 minutes (recommended implementation value).
- OTP verification attempt limit: 5 (recommended implementation value).
- Resend cooldown: 60 seconds (recommended implementation value).
- Email OTP rather than SMS OTP.
- Negative stock is blocked for outbound operations.
- Initial stock is optional and should be applied through a controlled initialization/stock adjustment path so it is auditable.
- A simple role-based access model is sufficient for the hackathon MVP.

20. Source Traceability

The product requirements in this PRD are based on the uploaded StockSense problem statement. The source defines the objective, target users, authentication flow, dashboard KPIs and filters, navigation, product management, receipts, deliveries, internal transfers, stock adjustments, low-stock alerts, multi-warehouse support, SKU search, and the simplified end-to-end inventory example.

[filecite\turn0file0\L2-L24](#) [filecite\turn0file0\L26-L59](#) [filecite\turn0file0\L61-L87](#)
[filecite\turn0file0\L88-L103](#)

Technical architecture and implementation details in this PRD are proposed to make the requirements actionable for an AI coding IDE. They are not claims that those technologies were mandated by the source document.

21. Final Build Order

- PHASE 1 — Foundation
- ☐ Create monorepo / frontend / backend
 - ☐ Configure TypeScript
 - ☐ Configure PostgreSQL
 - ☐ Configure Prisma
 - ☐ Create .env.example

- ☐ Configure Git and .gitignore

PHASE 2 — Database

- ☐ Implement Prisma schema
- ☐ Create migration
- ☐ Create seed data
- ☐ Verify relationships and constraints

PHASE 3 — Authentication

- ☐ Register/login
- ☐ JWT middleware
- ☐ Profile
- ☐ Brevo SMTP
- ☐ Forgot password
- ☐ OTP verification
- ☐ Password reset

PHASE 4 — Inventory Core

- ☐ Categories/UOM
- ☐ Products
- ☐ Warehouses/locations
- ☐ StockQuantity
- ☐ StockLedger
- ☐ Transaction-safe stock service

PHASE 5 — Operations

- ☐ Receipts
- ☐ Deliveries
- ☐ Internal transfers
- ☐ Adjustments
- ☐ Move history

PHASE 6 — Dashboard

- ☐ KPIs
- ☐ Filters
- ☐ Low-stock alerts
- ☐ Notifications

PHASE 7 — Frontend

- ☐ Layout/sidebar
- ☐ Auth pages
- ☐ Dashboard
- ☐ Product pages
- ☐ Operation pages
- ☐ Warehouse pages
- ☐ Profile

PHASE 8 — Quality

- ☐ Integration tests
- ☐ Error states
- ☐ Security review
- ☐ Seed/demo verification
- ☐ README

PHASE 9 — Deployment

- ☐ Cloud PostgreSQL
- ☐ Cloud backend
- ☐ Cloud frontend
- ☐ Environment variables
- ☐ Production smoke test

